【扔物线 Compose 课程讲义】5.6-PointerInputModifier 的功能介绍和原理简析

关于讲义

讲义是对课程内容中**所有结论的集中汇总**。所以在看完每节课程之后，迅速来看一遍讲义，对课程内容做回顾，会有非常好的吸收效果。

面试前不要背讲义

在面试之前，通过快速浏览一遍讲义来确认所有知识都没有忘记，是个很好的备战方式。

但**不要用【背讲义上的知识点】的方式来备战**。最有效的浏览方式是：

浏览讲义中的所有内容，并：

- 看到「懂，并且记得背后的原因（或技术细节）」的，直接跳过；
- 看到「懂，但细节已经忘了」的，快速回看当节课程，把知识捡起来；
- 看到「已经不太懂 / 完全忘了」的，认真重看当节课程，把知识学回来。

疑问或意见

如果你发现讲义或课程有错误，或有别的什么意见，可以向任何一个你认识的助教反馈。

如果认可我们的课程，还请给个好评！ 评价链接：https://rengwuxian.com/cp

`PointerInputModifier` 的写法

`Modifier.clickable()`

最简单、最上层的间接调用。

扔物线学堂 - 秉心 (1437182)

Kotlin 复制代码

```kotlin
Modifier.clickable {
  // 回调代码
}
```

`Modifier.combinedClickable()`

比 `clickable()` 功能多一些，也是最上层的。

Kotlin 复制代码

```kotlin
Modifier.combinedClickable(onLongClick = {
  // 回调代码
}, onDoubleClick = {
  // 回调代码
}) {
  // 回调代码（点击）
}
```

`Modifier.pointerInput()`

基本写法

在里面直接调用手势侦测函数：

Kotlin 复制代码

```kotlin
Modifier.pointerInput(Unit) {
  detectTapGestures(onTap = {
    // 回调代码
  }, onDoubleTap = {
    // 回调代码
  }, onPress = {
    // 回调代码
  }, onLongPress = {
    // 回调代码
  })
}
```

> 注：只作为举例。`detectTapGestures()` 并不是唯一的「直接调用手势侦测」的函数。

自主侦测

```kotlin
Modifier.pointerInput(Unit) {
  // forEachGesture() 和 awaitPointerEventScope() 并不是规定的写法，但几乎只能这么写
  forEachGesture {
    awaitPointerEventScope {
      // 侦测代码
    }
  }
}
```

`PointerInputModifier` 原理

- 和 `DrawModifier` 在 `LayoutNode` 的 `modifier` 遍历过程中一样，都会用从左到右的顺序放进固定的链表中，并和右边最近的 `LayoutModifier` 放在一起，触摸的范围和位置也由右边的 `LayoutModifier` 决定。
- 和同一个 `LayoutModifier` 保存在一起（即它左边）的多个 `PointerInputModifier`，左边的担当右边的「父 `Modifier` 」的角色。